Flow and Rate Control

Window Flow Control · Token Bucket · Rate Control · Max-Min · Proportional Fair

2026-06-08

1 : **2**

2 Window Flow Control **2**
2.1 2
2.2 4
2.3 4

3 Rate Control — Token(Leaky) Bucket **5**
3.1 5
3.2 ( ) 5
3.3 ( ) 5
3.4 (balance →) 6
3.5 p_0: (conservation) 7
3.6 permit 7
3.7 W trade-off 8
3.8 ( ) 8

4 Rate Control **10**
4.1 10
4.2 Overflow 10
4.3 (KKT) 10
4.4 — 11
4.5 3- - ( ) 11

5 Max-Min Fairness **13**
5.1 13
5.2 Bottleneck 14

5.3 Water-filling 14
5.4 14

6 Proportional Fair Scheduling **15**
6.1 15
6.2 proportional fair 15
6.3 3- + Multiuser diversity 16

7 **16**

1 :

“ , ” .

Window flow control	(outstanding) W	round-trip
Rate control (token bucket)	permit r, W	Markov chain ·
/ rate control	r_w	, max-min / proportional fair ·

, (·) .

!

, . [ ] “ ” .

---

2 Window Flow Control

2.1

End-to-end window ACK W .

- d: round-trip delay (sec)
- X: (sec) =
- W: window size ( )

window d , W window W/d, $1/X$. :

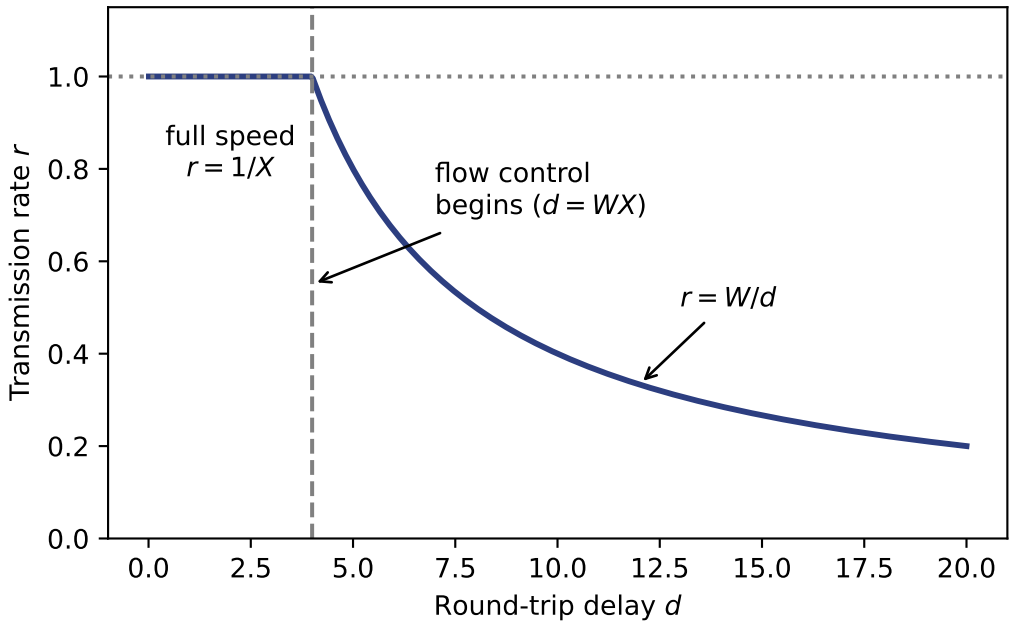
$$r = \min\left[\frac{1}{X},\ \frac{W}{d}\right] \quad \text{[packets/sec]}$$

- $d \le WX$: $r = 1/X$ (full speed, window)
- $d > WX$: $r = W/d$ (window throttle = flow control)

```
import numpy as np
import matplotlib.pyplot as plt

X, W = 1.0, 4.0
d = np.linspace(0.01, 20, 500)
r = np.minimum(1/X, W/d)

fig, ax = plt.subplots()
ax.plot(d, r, lw=2.2, color="#2c3e80")
ax.axvline(W*X, ls="--", color="gray")
ax.axhline(1/X, ls=":", color="gray")
ax.annotate("full speed\n$r=1/X$", xy=(W*X/2, 1/X),
            xytext=(W*X/2, 0.78), ha="center")
ax.annotate("$r=W/d$", xy=(12, W/12), xytext=(13, 0.5),
            arrowprops=dict(arrowstyle="->"))
ax.annotate("flow control\nbegins ($d=WX$)", xy=(W*X, 0.55),
            xytext=(7, 0.7), arrowprops=dict(arrowstyle="->"))
ax.set_xlabel("Round-trip delay $d$"); ax.set_ylabel("Transmission rate $r$")
ax.set_ylim(0, 1.15); plt.tight_layout()
```



1: RTT(d) . $d = WX$ full-speed flow control .

2.2

.

1. . d W $r = W/d$ → window RTT ( ).
2. . full speed $W \geq d/X = d \cdot$ (link rate), **bandwidth–delay product** in-flight .

2.3

- **Node-by-node (hop-by-hop):** window → **backpressure** , end-to-end .
- **Isarithmic:** permit → .

[ ] Window

- : $r = \min[1/X,\ W/d]$.
- : d, X, W r , full-speed $d \leq WX$, .
- “ ”, “BDP ” .

3 Rate Control — Token(Leaky) Bucket

. → → DTMC → → .

3.1

- **permit(token)** .
- Token $1/r$ **1** (W) . token .
- token .

3.2 ( )

token embedding Markov chain. i:

- $i = 0, 1, ..., W$: , token $(W - i)$. ($i = 0 =$, $i = W =$ token 0)
- $i = W + 1, W + 2, ...$: token 0 , $(i - W)$.

λ Poisson slot($1/r$) k

$$a_k = \frac{(\lambda/r)^k}{k!}\, e^{-\lambda/r}, \qquad k = 0, 1, 2, ...$$

$\lambda/r = \rho$ Poisson.

3.3 ( )

slot token -1, $+A$. $j \geq 1$ token $i = j - 1 + A$, $A = i - j + 1$:

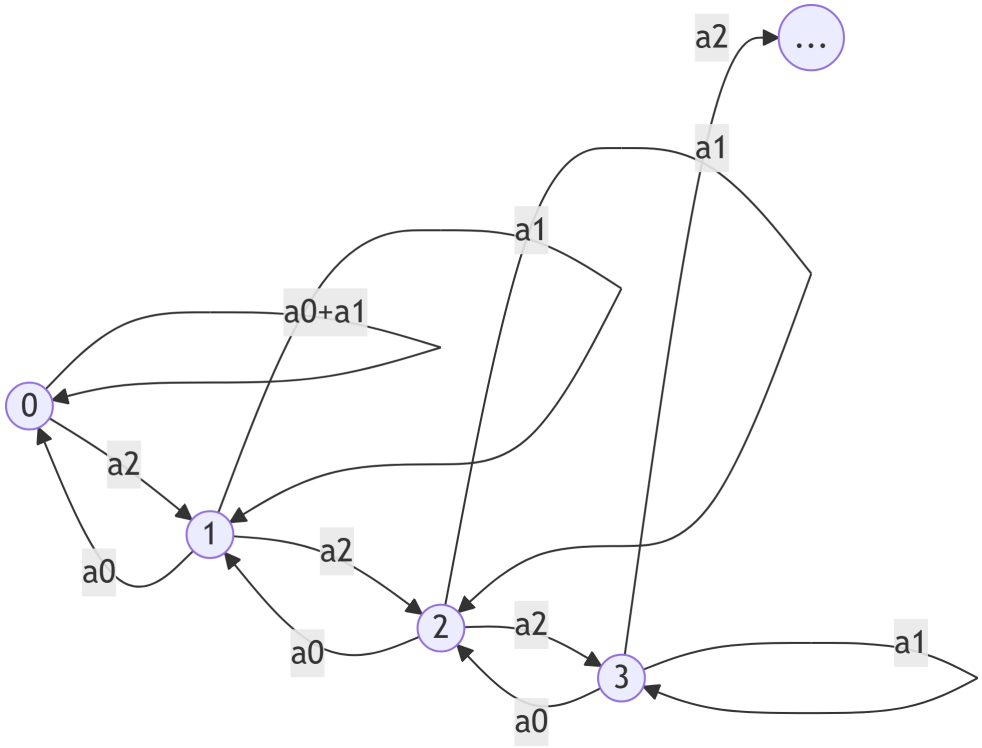
$$P_{ji} = a_{\,i-j+1} \quad (j \leq i + 1), \qquad 0 \text{ otherwise.}$$

0( ) : $A = 0$(token) $\cdot$ $A = 1$( ) 0 :

$$P_{0i} = \begin{cases} a_0 + a_1, & i = 0 \\ a_{\,i+1}, & i \geq 1 \end{cases}$$

$a_0 + a_1$ W . M/G/1 chain *skip-free to the left* .

```
flowchart LR
    S0((0)) -->|a0+a1| S0
    S0 -->|a2| S1((1))
    S1 -->|a0| S0
    S1 -->|a1| S1
    S1 -->|a2| S2((2))
    S2 -->|a0| S1
    S2 -->|a1| S2
    S2 -->|a2| S3((3))
    S3 -->|a0| S2
    S3 -->|a1| S3
    S3 -->|a2| S4(("..."))
```



2: Token bucket DTMC ( ). a_0, / $a_1, a_2, ...$

3.4 (balance →)

$j = 0$ $a_{i+1} = a_{i-0+1}$,

$$p_0 = a_0 p_1 + (a_0 + a_1) p_0, \qquad p_i = \sum_{j=0}^{i+1} a_{\,i-j+1}\, p_j \quad (i \geq 1).$$

0

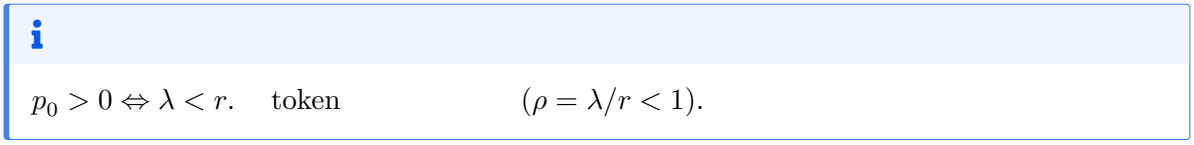
$$p_0(1 - a_0 - a_1) = a_0 p_1 \;\Rightarrow\; p_1 = \frac{1 - a_0 - a_1}{a_0}\, p_0,$$

$p_2, p_3, \cdots$ p_0 .

3.5 p_0: (conservation)

. Token r , **0** (a_0) $= r(1 - p_0 a_0)$. throughput $= \lambda$:

$$r(1 - p_0 a_0) = \lambda \;\Rightarrow\; \boxed{p_0 = \frac{r - \lambda}{r\, a_0}}$$

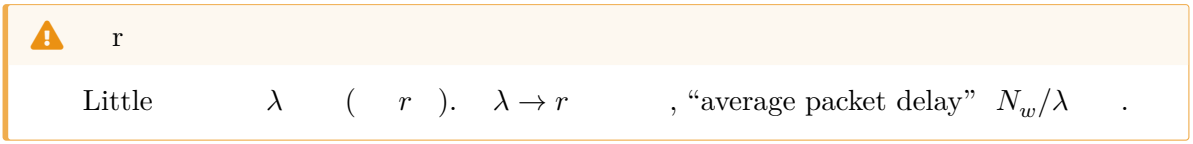


$p_0 > 0 \Leftrightarrow \lambda < r$. token $(\rho = \lambda / r < 1)$.

3.6 permit

$= \max(0, i - W)$, $N_w = \sum_{j>W} p_j (j - W)$. **Little** $N_w = \lambda T$:

$$\boxed{T = \frac{1}{\lambda} \sum_{j=W+1}^{\infty} p_j\, (j - W)}$$



r

Little λ (r). $\lambda \to r$, “average packet delay” N_w / λ .

3.7 W trade-off

- W → permit (,)
- W → ( )

r , W .

3.8 ( )

: $W = 0$(token), slot 2 .

$$a_0 = 0.4, \quad a_1 = 0.4, \quad a_2 = 0.2, \qquad \rho = \mathbb{E}[A] = 0.4 + 2(0.2) = 0.8 < 1.$$

$W = 0$ $n =$.

Balance ( )

$$\begin{aligned} p_0 &= (a_0 + a_1)p_0 + a_0 p_1 = 0.8p_0 + 0.4p_1 && \Rightarrow\ p_1 = 0.5\,p_0 \\ p_1 &= a_2 p_0 + a_1 p_1 + a_0 p_2 = 0.2p_0 + 0.4p_1 + 0.4p_2 && \Rightarrow\ p_2 = 0.25\,p_0 \\ p_2 &= a_2 p_1 + a_1 p_2 + a_0 p_3 && \Rightarrow\ p_3 = 0.125\,p_0 \end{aligned}$$

0.5 : $p_n = p_0(0.5)^n$. $\sum p_n = 2p_0 = 1 \to p_0 = 0.5$,

$$p_n = (0.5)^{n+1}.$$

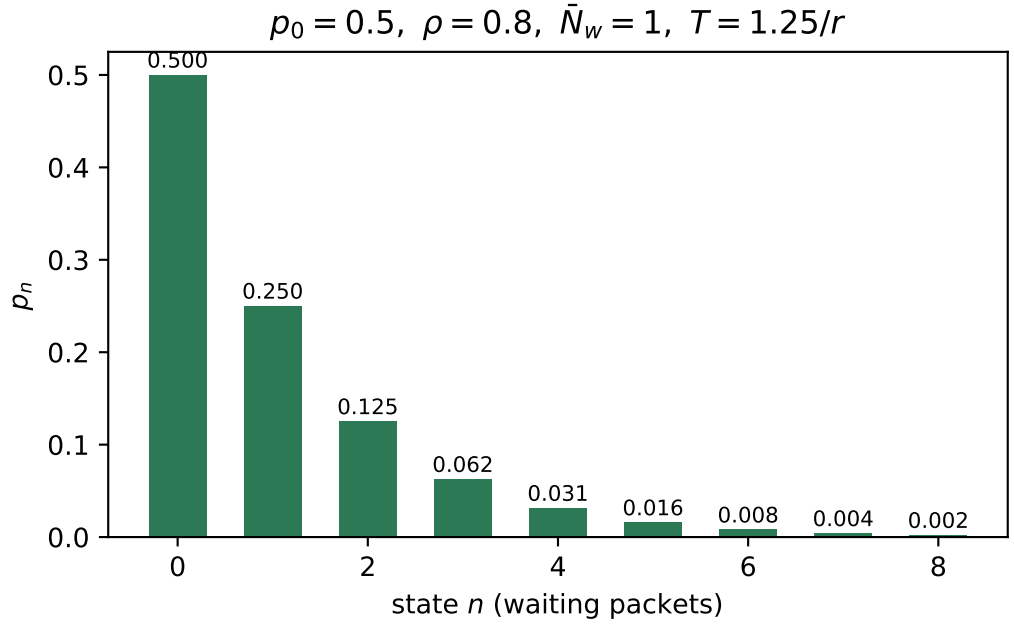
: $p_0 = \dfrac{r - \lambda}{r a_0} = \dfrac{1 - \rho}{a_0} = \dfrac{0.2}{0.4} = 0.5.$

. $W = 0$ $N_w = \mathbb{E}[n] = \sum_n n\,(0.5)^{n+1} = 1$. Little :

$$T = \frac{N_w}{\lambda} = \frac{1}{0.8\,r} = \frac{1.25}{r} \quad (= 1.25 \text{ slot}).$$

```
import numpy as np, matplotlib.pyplot as plt
n = np.arange(0, 9)
p = 0.5**(n+1)
fig, ax = plt.subplots()
ax.bar(n, p, color="#2c7a55", width=0.6)
for xi, pi in zip(n, p):
    ax.text(xi, pi+0.008, f"{pi:.3f}", ha="center", fontsize=8)
ax.set_xlabel("state $n$ (waiting packets)"); ax.set_ylabel("$p_n$")
```

```
ax.set_title(r"$p_0=0.5,\ \rho=0.8,\ \bar N_w=1,\ T=1.25/r$")
plt.tight_layout()
```



3: $p_n = (0.5)^{n+1}$ ( ).

[ ] Token bucket —

1. (token).
2. $P_{ji} = a_{i-j+1}$, $P_{00} = a_0 + a_1$ **MC** .
3. balance / .
4. $p_0 = \dfrac{r-\lambda}{r a_0}$.
5. N_w/λ (Little).

“ (token vs)” — slot . (a) · (c) .

4 Rate Control

4.1

class w $\bar{r}_w$, $r_w \le \bar{r}_w$:

$$\min \sum_{(i,j)} D_{ij}(F_{ij}) + \sum_w e_w(r_w) \quad \text{s.t.} \quad \sum_{p \in P_w} x_p = r_w,\ x_p \ge 0,\ 0 \le r_w \le \bar{r}_w.$$

- $\sum D_{ij}(F_{ij})$: (routing).
- $\sum e_w(r_w)$: (r_w → “ ”).

↑, ↑. .

4.2 Overflow

$y_w = \bar{r}_w - r_w \ge 0$ → :

$$\min \sum D_{ij}(F_{ij}) + \sum_w E_w(y_w), \quad \sum_p x_p + y_w = \bar{r}_w,\ x_p, y_w \ge 0,$$

$E_w(y_w) = e_w(\bar{r}_w - y_w)$, $e'_w(r_w) = -(a_w/r_w)^{b_w}$.

4.3 (KKT)

$d^*_p = \sum_{(i,j)\in p} D'_{ij}(F^*_{ij})$,

$$x^*_p > 0 \Rightarrow d^*_p \le d^*_{p'}\ (\forall p' \in P_w) \text{ and } d^*_p \le -e'_w(r^*_w),$$
$$r^*_w < \bar{r}_w \Rightarrow -e'_w(r^*_w) \le d^*_p\ (\forall p \in P_w).$$

: ( ) (= routing), $-e'_w$.

4.4 —

C, $\bar{r}$, r, $y = \bar{r} - r$. M/M/1 $D(r) = \dfrac{r}{C-r}$, $\dfrac{a}{r}$:

$$J(r) = \frac{r}{C-r} + \frac{a}{r}, \qquad J'(r) = \frac{C}{(C-r)^2} - \frac{a}{r^2}.$$

$J' = 0$:

$$\frac{\sqrt{C}}{C-r} = \frac{\sqrt{a}}{r} \Rightarrow r(\sqrt{C}+\sqrt{a}) = C\sqrt{a} \Rightarrow \boxed{r^* = \frac{C\sqrt{a}}{\sqrt{a}+\sqrt{C}}}$$

flow control ($r = \bar{r}$):

$$\frac{C}{(C-\bar{r})^2} < \frac{a}{\bar{r}^2} \iff \bar{r} < \frac{C\sqrt{a}}{\sqrt{a}+\sqrt{C}} = r^*.$$

$$r^* = \frac{C\sqrt{a}}{\sqrt{a}+\sqrt{C}} \cdot \frac{a}{a+C} \quad \frac{\sqrt{a}}{\sqrt{a}+\sqrt{C}} \quad .$$

4.5 3- - ( )

OD, C_1, C_2, C_3 one-hop , r :

$$\min \sum_{i=1}^{3} \frac{f_i}{C_i - f_i} \quad \text{s.t.} \ \sum_i f_i = r, \ f_i \geq 0.$$

Lagrangian $\mathcal{L} = \sum \frac{f_i}{C_i - f_i} - \nu(\sum f_i - r)$,

$$\frac{C_i}{(C_i - f_i)^2} = \nu \Rightarrow C_i - f_i = \sqrt{C_i/\nu} \Rightarrow f_i = C_i - \sqrt{\frac{C_i}{\nu}}.$$

$\sum f_i = r$:

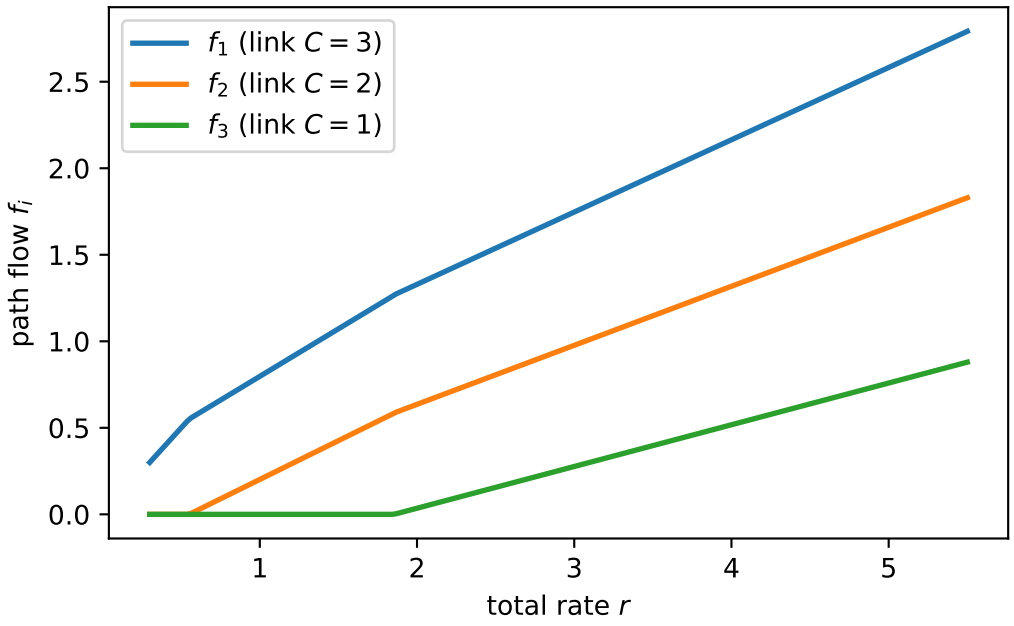
$$\sqrt{\nu} = \frac{\sum_i \sqrt{C_i}}{\sum_i C_i - r}, \qquad \boxed{f_i = C_i - \sqrt{C_i}\,\frac{\sum_j C_j - r}{\sum_j \sqrt{C_j}}}$$

($f_i < 0$ $f_i = 0$.)

```
import numpy as np, matplotlib.pyplot as plt

C = np.array([3.0, 2.0, 1.0])
def opt_flows(r):
    active = np.array([True, True, True])
    for _ in range(3):
        Ca = C[active]
        s = np.sum(np.sqrt(Ca)) / (np.sum(Ca) - r)
        f = np.zeros(3)
        f[active] = Ca - np.sqrt(Ca)/s
        if np.all(f[active] >= -1e-9):
            f[f < 0] = 0; return f
        active = f > 1e-9
    return np.clip(f, 0, None)

R = np.linspace(0.3, 5.5, 200)
F = np.array([opt_flows(r) for r in R])
fig, ax = plt.subplots()
for i, c in enumerate(C):
    ax.plot(R, F[:, i], lw=2, label=f"$f_{i+1}$ (link $C={c:g}$)")
ax.set_xlabel("total rate $r$"); ax.set_ylabel("path flow $f_i$")
ax.legend(); plt.tight_layout()
```



4: r . , . ($C = [3, 2, 1]$)

[ ] rate/routing

- : $\frac{C_i}{(C_i - f_i)^2}$ $+ \sum f_i = r$.
- / ( ).
- flow → , .

---

5 Max-Min Fairness

. · , r_p, $F_a = \sum_{p \ni a} r_p \le C_a$.

5.1

r feasible , r_p — $r_{p'} \le r_p$ — **max-min fair**.

: lexicographic .

5.2 Bottleneck

a p **bottleneck**: $F_a = C_a$ ( ) a p $(r_p \geq r_{p'})$.

Proposition: r max-min fair $\Leftrightarrow$ bottleneck .

5.3 Water-filling

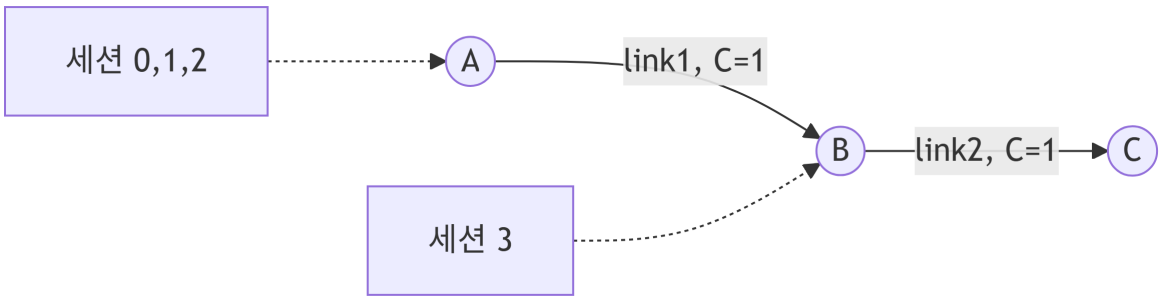
, .

: $k = 1,\ F_a^0 = 0,\ r_p^0 = 0,\ P^1 = P,\ A^1 = A.$

1. $n_a^k =$ a .
2. $\tilde{r}^k = \min_{a \in A^k} \dfrac{C_a - F_a^{k-1}}{n_a^k} \rightarrow$ bottleneck.
3. $r_p^k = r_p^{k-1} + \tilde{r}^k$ (p), .
4. $F_a^k = \sum_{p \ni a} r_p^k$.
5. $A^{k+1} = \{a : C_a - F_a^k > 0\}$.
6. $P^{k+1} = \{p : p$ $\}$.
7. $k \leftarrow k+1$. 8. P .

5.4

```
flowchart LR
    A((A)) -->|"link1, C=1"| B((B))
    B -->|"link2, C=1"| D((C))
    S012["  0,1,2"] -.-> A
    S3["  3"] -.-> B
```



5: (1). 0,1,2 , 3 .

$0 \cdot 1 \cdot 2$ → 1/3. 3 $1 - 1/3 = 2/3$ → $r_3 = 2/3$. max-min (
).

[ ] Max-Min

- ( ) bottleneck (iff) .
- **water-filling** .
- "max-min " — .

---

6 Proportional Fair Scheduling

. **throughput fairness** (max-min).

6.1

$\mathrm{DRC}_k(t)$, $R_k(t)$. slot

$$k^* = \arg\max_k \frac{\mathrm{DRC}_k(t)}{R_k(t)},$$

$$R_k(t+1) = \begin{cases} \left(1 - \frac{1}{t_c}\right) R_k(t) + \frac{1}{t_c} \mathrm{DRC}_k(t), & k \\ \left(1 - \frac{1}{t_c}\right) R_k(t), & k \end{cases}$$

" "
.

6.2 proportional fair

( ) :

$$P = \arg\max \sum_{k \in U} \log R_k.$$

$\partial(\log R_k)/\partial R_k = 1/R_k$ — 1 , . slot DRC_k/R_k $\sum \log R_k$
(gradient).

: $\sum_k \Delta R_k / R_k \le 0$ ( ).

6.3 3- + Multiuser diversity

throughput , ( ) . PF R_k **peak** →
fading throughput (**multiuser diversity**).

	Max-Min	Proportional Fair
	( )	( )
throughput		throughput–fairness

[ ] Proportional Fair

- $\sum \log R_k$ $\arg\max \mathrm{DRC}_k/R_k$, 3 “ ”
- max-min 1 (vs ,)

7

( ) → ( )

	( )
Window	$r = \min[1/X, W/d]$ · / full-speed BDP
Token bucket rate/routing	chain + Little **MC** p_0, T KKT ·
Max-Min	bottleneck iff **water-filling**
Proportional Fair	$\sum \log R_k$ + 3-

token bucket(3.8) (4.5) , · · . token bucket “ →
MC → balance → N_w/λ” 4 , “ + $\sum f_i = r$”